

Athena

Autonomous Product Intelligence & Decision Support System

Technical Report

Agentic RAG · Multi-agent deep research · Long-term memory
Google Gemini · Hybrid retrieval · Evaluation & observability

Athena — Technical Report

Project: Autonomous Product Intelligence & Decision Support System

1. Problem framing

SaaS companies drown in information (tickets, feedback, PRDs, meetings, issues, releases, interviews, competitor reports) but starve for insight. Answering real business questions requires connecting evidence across sources, reasoning over time, validating findings, and recommending action — beyond what search or a naive chatbot can do.

I built **Athena**, an autonomous product-intelligence analyst that plans, researches, validates, remembers, and reports, with every answer backed by traceable evidence.

2. Design decisions & rationale

2.1 Provider: Google Gemini (LLM and embeddings)

- **One key, one SDK** for generation + embeddings → simpler ops, generous free tier (important for a no-budget task), and a 1M-token context on Pro for synthesis over large evidence sets.
- Model routing ([config.py](#)): Flash for agents/reranking, Pro for synthesis and reports, Flash-Lite for bulk enrichment. Cost and latency scale with the job.

2.2 Advanced RAG: hybrid retrieval + rerank (not pure vector search)

- **Why hybrid?** The task's questions span *semantic* intent ("drivers of dissatisfaction") and *lexical* intent ("SSO", "SCIM", version strings). Pure dense retrieval misses rare exact tokens; pure BM25 misses paraphrase. I fuse both.
- **Why RRF for fusion?** Cosine similarity and BM25 live on incomparable scales. Reciprocal Rank Fusion needs only ranks, so it's robust and nearly parameter-free (one constant, [k=60](#)).
- **Why an LLM reranker?** Turning "top-40 by fusion" into "top-8 that actually answer the query" measurably raises faithfulness and shrinks the generation context (cheaper, less distraction). It's a pragmatic stand-in for a trained cross-encoder, with zero extra infra.
- **Asymmetric embeddings** ([RETRIEVAL_DOCUMENT](#) vs [RETRIEVAL_QUERY](#)) improve recall for free.

2.3 Agentic workflow: explicit state machine (not a black-box framework)

- I implemented Planner → Researcher(s) → Validator → Synthesizer as visible code with a streaming event API, rather than adopting an opaque agent framework.
- **Trade-off:** I give up some out-of-the-box tooling in exchange for transparency, debuggability, deterministic control flow, and per-step tracing — properties that matter far more for a *reliable, explainable* production analyst (and for defending the design in a review).
- The **Validator** is the key reliability primitive: findings are independently checked against their cited evidence, and unsupported claims are dropped before they can reach the answer. This makes "evidence-backed" and "validated" real guarantees, not marketing.

2.4 Knowledge management: a lightweight knowledge graph

- During ingestion I build an entity co-occurrence graph (entity■docs, entity■entity, entity■sentiment). It powers "which customers are most affected by X", "what connects these requests", and the aggregate statistics that seed reports. Simple (dict-of-counters, JSON-persisted) but delivers the value of a graph DB with none of the operational weight for a demo.

2.5 Long-term memory

- Research runs distill their top validated findings into a semantic memory store; new questions recall relevant memories to prime planning. Insight compounds across sessions — a differentiator from stateless chatbots.

2.6 Evaluation & observability as first-class features

- **Tracing** everything (spans with timing/tokens/IO) makes the system explainable and debuggable, and is surfaced in the UI.
- An **LLM-as-judge** eval harness plus reference-based metrics (possible because the corpus is deterministic and its "correct" sources are known) gives an honest, repeatable quality signal instead of vibes.

2.7 Reproducible synthetic data

- A deterministic generator (fixed seed, no API) produces 281 interconnected documents. **Determinism matters:** anyone can rebuild the identical corpus, making the demo and the eval numbers reproducible. Shared entities across sources are what make cross-source reasoning demonstrable.

3. Architecture

See [ARCHITECTURE.md](#) for the full component breakdown and data-flow diagrams. In brief:

Ingestion → (Chroma + BM25 + Graph) → Hybrid retrieval+rerank → {RAG QA | Multi-agent research | Reports} ↔ {Memory, Tracing/Eval} → Streamlit UI

4. Trade-offs made

Decision	Chosen	Gave up	Why it's right here
Vector store	Chroma (in-process)	Managed scale-out DB	Zero-infra, reproducible local demo; interface swappable.
Fusion	RRF	Learned fusion / weighted scores	Robust, tuning-free across incomparable scales.
Reranker	LLM judgment	Trained cross-encoder	No extra model/infra; good enough and explainable.
Agent runtime	Hand-rolled state machine	Framework features	Transparency, control, per-step tracing.
Graph	dict-of-counters	Neo4j	Same insight value, no service to run.
Data	Synthetic, deterministic	Real corpus	Reproducibility + guaranteed cross-source links.
Enrichment	Metadata-derived first	LLM on every chunk	Fast, free, deterministic indexing for the demo corpus.

5. Challenges faced & how I solved them

1. **Citation integrity across merged evidence.** Each Researcher retrieves its own evidence with local `E#` ids; naively merging would misalign citations. **Solution:** resolve each finding's `E#` to durable `chunk_ids` at creation time, then re-derive a consistent `E#` space over the merged evidence for the Validator and Synthesizer. No fabricated or dropped citations. (Covered by a round-trip unit test.)
2. **Comparing dense and sparse scores.** They're on different scales. **Solution:** rank-based RRF instead of score normalization.
3. **BM25 on tiny corpora returns zeros** (IDF collapses when a term is in half the docs). This surfaced in a unit test. **Solution:** understood it as an inherent BM25 property (not a bug), filtered non-positive scores, and made the test corpus realistically sized. Documented inline.
4. **Keeping the demo runnable without paying for indexing.** **Solution:** deterministic dataset + metadata-derived enrichment so only embeddings need the API, and the whole ingestion/retrieval-logic path is testable offline.

5. **Model landscape drift.** My training predates current Gemini IDs. **Solution:** verified current models at build time and centralized all IDs in `config.py` behind env overrides, so nothing hard-codes a name.
6. **Rate limits / transient API errors.** **Solution:** one central client with exponential-backoff retries and graceful degradation (agents never crash the pipeline on a bad LLM response — they fall back).

6. Evaluation approach

The eval harness ([athena/eval/](#)) scores a golden set of the task's example questions on:

- **retrieval_source_coverage** — did we retrieve the source types a good answer needs? (reference-based)
- **keyword_recall** — are must-mention entities present?
- **citation_coverage** — are factual paragraphs grounded with `[E#]`?
- **faithfulness** (LLM judge) — is every claim supported by evidence?
- **answer_relevance** (LLM judge) — does the answer address the question?

Results are aggregated, saved, and shown in the UI. (Run `python -m athena.scripts.run_eval` or use the Evaluation tab. Requires a key.)

7. Scalability considerations

- **Ingestion** is embarrassingly parallel and batched; embeddings already batch.
- **Retrieval** stores are swappable for horizontally-scaled services (pgvector, OpenSearch) behind the existing [KnowledgeBase](#) facade.
- **Agents** are stateless per request; the orchestrator can fan Researchers out concurrently (currently sequential for trace clarity — an easy async upgrade).
- **Memory/graph** move to Postgres/Neo4j with no change to callers.
- **Cost** scales down via model routing and the reranker shrinking generation context.

8. Future improvements

1. **Concurrency:** run Researchers in parallel (asyncio) for lower latency.
2. **Iterative research loop:** let the Validator trigger follow-up retrieval when findings are weak (the config already reserves `max_research_rounds`).
3. **Trained reranker / ColBERT** for retrieval quality at scale.
4. **Real connectors:** Zendesk, Jira, GitHub, Google Drive, Slack ingestion.
5. **Temporal reasoning:** first-class time-decay and trend detection over the `created_at` axis.
6. **Feedback loop:** capture thumbs-up/down to fine-tune retrieval + prompts.
7. **RBAC & multi-tenancy** for a real organizational deployment.
8. **FastAPI service layer** so the analyst is callable from Slack/other apps.

9. What I'd highlight in a review

- The **Validator-gated, citation-integral** research pipeline — reliability and explainability designed in, not bolted on.
- **Hybrid retrieval with RRF + rerank** chosen deliberately for the shape of the task's questions.
- End-to-end **observability + evaluation** treated as product features.
- A codebase that is **transparent and swappable** — every "demo" choice has a named production upgrade path.

Appendix: Architecture

This document describes the system design of Athena, the Autonomous Product Intelligence & Decision Support System.

1. Design goals

The task asks for a system that behaves like an **autonomous knowledge worker**, not a chatbot. Concretely it must: understand, retrieve, reason, plan, research, validate, remember, and report — with explainable, evidence-backed answers.

That shaped five guiding principles:

- 1. **Evidence-first.** Nothing is asserted without a retrievable source. Every factual claim is cited and resolvable back to its exact chunk.
- 2. **Transparent agency.** Agent control flow is explicit code with per-step tracing — not a black box. Reliability and explainability beat "magic".
- 3. **Hybrid retrieval.** Business questions mix semantic and lexical intent, so retrieval combines dense + sparse and reranks.
- 4. **Compounding knowledge.** Long-term memory and a knowledge graph let insight accumulate across sessions and connect across sources.
- 5. **Reproducible & runnable.** Local-first stores, a deterministic dataset, and one API key — anyone can run the whole thing.

2. System overview



3. Component design

3.1 Ingestion ([athena/ingestion/](#))

Stage	File	What it does
-------	------	--------------

Generate	<code>generate_dataset.py</code>	Deterministic synthetic corpus for a fictional SaaS ("Flowdesk"): 281 docs across 8 source types, spanning ~12 months, with shared entities so cross-source reasoning has real signal.
Load	<code>loaders.py</code>	Reads JSON (structured) or <code>.md/.txt</code> (user drop-ins) into typed <code>Documents</code> .
Chunk	<code>chunker.py</code>	Paragraph-aware, size-bounded chunking with overlap. Respects document structure (headings/sections stay together).
Enrich	<code>enrich.py</code>	Attaches entities, themes, sentiment. Derives from structured metadata when present (free, deterministic); falls back to an LLM extractor for unstructured inputs.

3.2 Knowledge store & retrieval (`athena/retrieval/`)

- `vector_store.py` — Chroma persistent client. We embed with Gemini (`gemini-embedding-001`) using **asymmetric task types** (`RETRIEVAL_DOCUMENT` for indexing, `RETRIEVAL_QUERY` for search), which improves recall vs. symmetric embeddings. Chroma stores vectors + metadata and does cosine ANN.
- `sparse_index.py` — BM25 (rank-bm25). Captures exact tokens dense retrieval misses: "SSO", "SCIM", "SAML", error codes, customer/version strings.
- `knowledge_graph.py` — co-occurrence graph of entities ■ documents, entity ■ entity, and entity ■ sentiment. Powers "connect the dots" queries and aggregate report statistics.
- `hybrid.py` — the retrieval pipeline:
 - dense search + sparse search (candidates)
 - **Reciprocal Rank Fusion** (rank-based, so incomparable score scales don't matter)
 - **LLM cross-encoder rerank** (Gemini flash) to pick the top-N truly relevant
 - metadata filtering by `source_type` (from agent hints)
- `knowledge_base.py` — a single facade (`build()` / `load()` / `retrieve()`) used by everything downstream.

3.3 RAG QA (`athena/agents/rag.py`)

Single-shot path for simple questions: retrieve → build cited evidence set → grounded generation. Resolves `[E#]` citations in the answer back to source labels. Used by the **Chat** tab and the **evaluation** harness.

3.4 Multi-agent deep research (`athena/agents/`)

An explicit state machine:

RECALL MEMORY → PLAN → RESEARCH (per sub-question) → VALIDATE → SYNTHESIZE → REMEMBER

Agent	File	Role
Planner	<code>planner.py</code>	Decomposes the question into ≤6 researchable sub-questions with source hints; primed with recalled memory.
Researcher	<code>researcher.py</code>	Retrieves evidence per sub-question and produces specific, cited findings. Runs independently per sub-question.
Validator	<code>validator.py</code>	Independently checks each finding against its evidence; drops unsupported claims, flags partial ones. This is the anti-hallucination gate.

Synthesizer	synthesizer.py	Weaves validated findings into a coherent, cited answer with recommended actions (uses Gemini Pro for long-context reasoning).
Orchestrator	orchestrator.py	Drives the state machine, merges evidence, emits streaming progress events, and traces every step.

Citation integrity: Researchers cite evidence with local `E#` ids, which are immediately resolved to **durable `chunk_ids`**. When evidence from all sub-questions is merged, findings remain valid and the synthesizer/validator see a consistent `E#` space re-derived from the merged set. No citation is ever fabricated or silently dropped.

3.5 Long-term memory ([athena/memory/long_term.py](#))

JSONL-backed store + in-memory embedding index. Stores distilled findings from research runs and user preferences. `recall()` does semantic top-k retrieval and feeds the planner, so discoveries compound across sessions.

3.6 Reports ([athena/reports/generator.py](#))

Four report types (executive, trends, customer, product). Each combines knowledge-graph **aggregate stats** + retrieved **evidence** and asks Gemini Pro for a polished, cited, leadership-ready Markdown document.

3.7 Evaluation & observability ([athena/eval/](#), [athena/core/tracing.py](#))

- **Tracing:** every operation is a span (llm/retrieval/agent) with timing, inputs/outputs, and token estimates, persisted as JSON and shown in the UI's **Traces** tab. Zero external dependencies.
- **Eval harness:** a golden set of business questions scored on retrieval source coverage, keyword recall, citation coverage, and **LLM-as-judge** faithfulness & answer relevance. Aggregated into a report shown in the **Evaluation** tab.

3.8 UI ([athena/ui/app.py](#))

Streamlit app with six tabs: Chat, Deep Research (live agent progress), Reports, Knowledge Explorer (graph), Evaluation, and Traces.

4. Model routing

Task	Model	Why
Agents, retrieval reranking, RAG answers	gemini-2.5-flash	Fast, cheap, strong for agentic/tool reasoning.
Synthesis & report writing	gemini-2.5-pro	Best long-context reasoning for connecting insights.
Bulk metadata enrichment	gemini-2.5-flash-lite	Cheapest for high-volume extraction.
Embeddings	gemini-embedding-001	One provider for LLM + vectors; asymmetric task types.

All IDs are centralized in [athena/core/config.py](#) and overridable via env vars.

5. Data flow for a deep-research question

1. User asks a complex question in the **Deep Research** tab.
2. Orchestrator **recalls** related memories → primes the Planner.
3. Planner returns sub-questions (+ source hints).
4. Each Researcher **hybrid-retrieves** evidence and emits cited findings.
5. Evidence is merged & de-duplicated across sub-questions.

6. Validator judges each finding against evidence; unsupported ones are dropped.
7. Synthesizer composes the final cited answer + recommended actions.
8. Top findings are **written to memory**; the full run is **traced** to disk.
9. UI streams every step live and shows the answer, findings, and sources.

6. Extensibility / production swaps

The interfaces are intentionally thin so demo-grade components swap for production ones without touching business logic:

Demo	Production option
Chroma (in-process)	pgvector / Qdrant / Pinecone
BM25 pickle	Elasticsearch / OpenSearch
JSONL memory	Postgres / a vector DB namespace
Co-occurrence graph (dict)	Neo4j
File traces	OpenTelemetry / Langfuse
Streamlit	FastAPI + React

See [TECHNICAL_REPORT.md](#) for trade-offs and future work.